Exploradores NoSQL

Modelo documental en MongoDB para el subsistema de hospitalización de la HCE

Curso: Bases de Datos II (G100), Unidad 3

Docente: Oralia Cortés Grajales

Institución Universitaria Pascual Bravo, Tecnología en Desarrollo de Software

Estudiante: Jhonatan Steven Ciro Amin

Fecha: noviembre de 2026

1. Introducción

Este informe presenta el trabajo realizado sobre el subsistema de hospitalización de la Historia Clínica Electrónica (HCE) de la Gobernación de Antioquia, ahora con un modelo NoSQL documental. Se eligió MongoDB porque la información de una hospitalización (evolución clínica, signos vitales y prescripciones) crece en el tiempo y se consulta casi siempre junto con la hospitalización, lo que encaja bien con documentos anidados. El modelo relacional construido antes sigue siendo útil para la información estable y muy normalizada; MongoDB complementa ese modelo en la parte de historia clínica con alto volumen y estructura variable.

2. Descripción del modelo documental

La base de datos se llama hce_hospitalizacion y tiene seis colecciones. Cinco son de referencia (pacientes, medicos, camas, diagnosticos y medicamentos) y una es la colección principal (hospitalizaciones). Los _id de las colecciones de referencia son códigos legibles (por ejemplo P001, M001, C101, I10, MED01), y en el caso de diagnosticos se usa el código CIE-10.

Decisión de diseño: qué se embebe y qué se referencia. Dentro de cada documento de hospitalizaciones se embeben la evolución clínica (arreglo de notas con fecha y signos vitales) y las prescripciones (arreglo con medicamento_id, dosis, frecuencia y días), porque no tienen sentido fuera de su hospitalización y siempre se leen con ella. En cambio, paciente, médico, cama, diagnóstico y medicamento se referencian por su identificador, porque son entidades con vida propia que se reutilizan en muchas hospitalizaciones y cuyos datos no conviene duplicar. Las consultas que necesitan los datos de esas entidades usan $lookup.

Estructura de un documento de hospitalizaciones (resumida):

```
{
  "_id": "H013", "paciente_id": "P013", "medico_id": "M003", "cama_id": "C103",
  "fecha_ingreso": ISODate("2026-11-03T..."), "fecha_egreso": null, "estado": "activa",
  "diagnostico_principal": "J18", "diagnosticos_secundarios": ["J44"],
  "evolucion": [ { "fecha": ISODate(...), "nota": "...",
                   "signos": { "fc": 88, "pa_sistolica": 120, "pa_diastolica": 80, "temp": 37.1, "spo2": 94 } } ],
  "prescripciones": [ { "medicamento_id": "MED05", "dosis": "dosis estandar IV",
                        "frecuencia": "cada 8 horas", "dias": 7 } ],
  "eliminado": false
}
```

La colección hospitalizaciones tiene validación de esquema con $jsonSchema (campos obligatorios, tipos de fecha, estado limitado a activa o egresada, rango de la saturación de oxígeno entre 0 y 100) e índices por paciente_id, por médico y estado, y por diagnóstico principal. La colección pacientes tiene un índice único por documento. Los datos simulados son coherentes: 14 pacientes, 6 médicos, 10 camas, 8 diagnósticos CIE-10, 14 medicamentos y 16 hospitalizaciones (10 egresadas y 6 activas, cada activa en una cama distinta marcada como ocupada). Los medicamentos prescritos corresponden al diagnóstico principal (por ejemplo, ceftriaxona y azitromicina para neumonía). Los datos se generan con una semilla fija en el script scripts/generar_datos.py y quedan en la carpeta datos/.

3. Operaciones CRUD

Los scripts de mongosh están numerados en el repositorio y se ejecutan en orden:

Create (02_insertar.js): carga las seis colecciones con insertMany, y luego inserta un paciente nuevo (P015) con su contacto anidado, una hospitalización nueva (H017) con evolución y prescripción embebidas, y marca la cama C107 como ocupada.

Update (03_actualizar.js): se actualiza un campo simple (eps del paciente), un campo anidado con notación de punto (contacto.telefono), se agrega una evolución con $push, se corrige un campo dentro de un elemento del arreglo con el operador posicional ($), se incrementan los días de ceftriaxona en todas las hospitalizaciones activas con $inc y arrayFilters, y se registra un egreso (estado, fecha de egreso y liberación de la cama).

Delete (04_eliminar.js): se hace un soft delete de la hospitalización H001 (eliminado: true, fecha y motivo), porque una historia clínica no debería borrarse físicamente; se quita una prescripción con $pull; y solo para los datos de prueba creados en el script se hace eliminación definitiva con deleteOne. Las consultas filtran siempre por eliminado: false.

4. Diseño de consultas avanzadas

Se diseñaron cinco pipelines con el Aggregation Framework, cada uno de 3 o 4 etapas. El código completo está en consultas/pipelines.js.

Q1. Estancia promedio por diagnóstico

Muestra cuántos días dura en promedio la hospitalización según el diagnóstico principal, solo para egresos no eliminados. Sirve para planear la ocupación de camas. Usa $subtract entre dos fechas (el resultado queda en milisegundos) y $divide para pasarlo a días.

```
coleccion: "hospitalizaciones",
    pipeline: [
      { $match: { estado: "egresada", eliminado: false } },
      { $group: {
          _id: "$diagnostico_principal",
          egresos: { $sum: 1 },
          estancia_promedio_ms: { $avg: { $subtract: ["$fecha_egreso", "$fecha_ingreso"] } }
      } },
      { $project: {
          _id: 0, diagnostico: "$_id", egresos: 1,
          estancia_promedio_dias: { $round: [{ $divide: ["$estancia_promedio_ms", 86400000] }, 1] }
      } }
    ]
```

Resultado (fragmento): A09: 2 egresos, 5.5 días. E11: 1 egreso, 9.0 días. I10: 1 egreso, 3.0 días. I50: 1 egreso, 3.0 días. J18: 1 egreso, 7.0 días. J44: 2 egresos, 6.0 días. K35: 1 egreso, 5.0 días. N39: 1 egreso, 7.0 días.

Q2. Carga de pacientes activos por médico

Cuenta las hospitalizaciones activas de cada médico y trae su nombre y especialidad con $lookup. Sirve para repartir la carga asistencial.

```
coleccion: "hospitalizaciones",
    pipeline: [
      { $match: { estado: "activa", eliminado: false } },
      { $group: { _id: "$medico_id", pacientes_activos: { $sum: 1 } } },
      { $lookup: { from: "medicos", localField: "_id", foreignField: "_id", as: "medico" } },
      { $project: {
          _id: 0, medico_id: "$_id", pacientes_activos: 1,
          nombre: { $arrayElemAt: ["$medico.nombre", 0] },
          especialidad: { $arrayElemAt: ["$medico.especialidad", 0] }
```

```
      } }
    ]
```

Resultado (fragmento): M003 Dr(a). Pineda (Neumología): 3 pacientes activos. M005 Dr(a). Escobar (Cirugía general): 2. M006 Dr(a). Montoya (Pediatría): 1.

Q3. Medicamentos más prescritos

Desanida el arreglo de prescripciones con $unwind, cuenta y ordena con $sortByCount y trae el nombre y la clase con $lookup. Sirve para planear el inventario de farmacia.

```
coleccion: "hospitalizaciones",
    pipeline: [
      { $unwind: "$prescripciones" },
      { $sortByCount: "$prescripciones.medicamento_id" },
      { $lookup: { from: "medicamentos", localField: "_id", foreignField: "_id", as: "med" } },
      { $project: {
          _id: 0, medicamento_id: "$_id", prescripciones: "$count",
          nombre: { $arrayElemAt: ["$med.nombre", 0] },
          clase: { $arrayElemAt: ["$med.clase", 0] }
      } }
    ]
```

Resultado (fragmento): Ceftriaxona (Antibiótico): 6 prescripciones. Enalapril (Antihipertensivo): 4. Los demás medicamentos aparecen con 2 prescripciones cada uno.

Q4. Alerta por baja saturación de oxígeno

Encuentra hospitalizaciones activas con alguna evolución con SpO2 menor a 92, y devuelve solo las evaluaciones críticas (con $filter y $map) junto con el nombre del paciente. Sirve para priorizar la revisión médica.

```
coleccion: "hospitalizaciones",
    pipeline: [
      { $match: { estado: "activa", eliminado: false, "evolucion.signos.spo2": { $lt: 92 } } },
      { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "paciente" } },
      { $project: {
          _id: 1, cama_id: 1, diagnostico_principal: 1,
          paciente: { $concat: [
            { $arrayElemAt: ["$paciente.nombres", 0] }, " ", { $arrayElemAt: ["$paciente.apellidos", 0] }
          ] },
          evaluaciones_criticas: { $map: {
            input: { $filter: { input: "$evolucion", as: "e", cond: { $lt: ["$$e.signos.spo2", 92] } } },
            as: "c", in: { fecha: "$$c.fecha", spo2: "$$c.signos.spo2" }
          } }
      } }
    ]
```

Resultado (fragmento): H014 (cama C104, J18): Felipe Londono, SpO2 de 90 el 2026-11-04. H015 (cama C105, I50): Maria Gomez, SpO2 de 88 el 2026-10-31.

Q5. Casos por municipio y diagnóstico (reporte epidemiológico)

Une cada hospitalización con el paciente, desanida el resultado y agrupa por municipio de residencia y diagnóstico principal. Sirve para detectar dónde se concentran ciertos diagnósticos. El resultado tiene 14 grupos; ejemplos:

```
coleccion: "hospitalizaciones",
    pipeline: [
      { $match: { eliminado: false } },
      { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "paciente" } },
      { $unwind: "$paciente" },
      { $group: {
          _id: { municipio: "$paciente.municipio", diagnostico: "$diagnostico_principal" },
```

```
        casos: { $sum: 1 }
    } }
]
```

Resultado (fragmento): Bello / A09: 1 caso. Bello / E11: 2 casos. Bello / I10: 1 caso. Caucasia / J44: 1 caso. Rionegro / J18: 2 casos.

Nota: el orden de las filas de Q2, Q4 y Q5 puede variar al ejecutar los pipelines, porque no incluyen $sort.

5. Evidencia de uso de IA

Se usó un asistente de IA (Claude) como apoyo en el desarrollo del trabajo: para proponer el diseño del modelo documental, escribir los scripts de mongosh, generar los datos simulados y revisar los pipelines. A continuación se resumen las interacciones.

Interacción 1. Prompt: se entregó la guía de la tarea y se pidió desarrollar todo (modelo, scripts, consultas, informe y guion del video). Resultado: propuesta de un modelo con cinco colecciones de referencia y una colección principal de hospitalizaciones con la evolución clínica y las prescripciones embebidas, datos simulados con semilla fija y cinco scripts numerados de mongosh (creación, inserción, actualización, eliminación y consultas).

Interacción 2. Prompt: comprobar que los pipelines funcionaran. Resultado: se ejecutaron en un simulador en memoria (mongomock). Los pipelines Q2, Q4 y Q5 corrieron correctamente. Q1 y Q3 usan $round y $sortByCount, que el simulador no implementa, y sus resultados se calcularon aparte en Python con los mismos datos.

Interacción 3. Prompt: ajustar el informe para un solo integrante y mejorar su organización. Resultado: se reorganizaron las secciones y se corrigió el texto.

Reflexión sobre el uso de IA: la IA aceleró la escritura de scripts y datos, pero cada consulta se revisó para entender qué hace cada etapa, porque la IA puede generar código con errores o que no se ha ejecutado en el entorno donde se va a usar.

6. Reflexión ética sobre la manipulación de datos clínicos

Los datos clínicos son datos sensibles: revelan estado de salud, y su mal uso puede causar discriminación, estigmatización o pérdida de confianza en el sistema de salud. En Colombia están protegidos por la Ley 1581 de 2012 (protección de datos personales), que da trato especial a los datos de salud, y la historia clínica tiene reserva legal según la Resolución 1995 de 1999. Por eso, en este trabajo todos los datos son simulados y no corresponden a personas reales.

En un sistema real, el modelo documental exige cuidados adicionales. Primero, el acceso por roles (médico tratante, administrativo, auditor), de modo que cada persona vea solo lo necesario, como se hizo en la tarea relacional. Segundo, no borrar físicamente las historias clínicas: el soft delete conserva la trazabilidad y permite auditar quién cambió qué. Tercero, la flexibilidad de MongoDB es una ventaja pero también un riesgo: sin validación de esquema se pueden guardar datos incompletos o inconsistentes, por eso se usó $jsonSchema. Cuarto, los reportes epidemiológicos como Q5 deben anonimizarse o agregarse para que no se pueda identificar a una persona en municipios pequeños. Finalmente, el uso de IA con datos clínicos reales no debe hacerse con datos identificables, porque se

compartiría información sensible con un tercero.

7. Conclusiones

El modelo documental permite leer una hospitalización completa con una sola consulta, y el Aggregation Framework resuelve reportes clínicos y epidemiológicos con pocas etapas. A cambio, las relaciones entre entidades hay que resolverlas con $lookup o con duplicación controlada, y la integridad no la garantiza el motor como en el modelo relacional. Por eso la propuesta es híbrida: el modelo relacional para los datos maestros y transaccionales, y MongoDB para la historia clínica de alto volumen y consultas de análisis.

8. Repositorio y video

Repositorio. Contiene los scripts de creación del modelo, inserción, actualización, eliminación y consultas (carpeta scripts), las cinco consultas avanzadas (carpeta consultas), los datos de prueba en JSON (carpeta datos), las capturas de ejecución (carpeta capturas) y este informe (carpeta informe). Un archivo README explica el orden de ejecución.

Video demostrativo. Dura máximo 10 minutos y muestra el modelo, la carga de datos, las operaciones CRUD (insertar, actualizar y eliminar) y la ejecución de las consultas avanzadas, explicando qué hace cada etapa de los pipelines.